

Java Collection Framework – Collection Classes

Aim / Objective

To implement and demonstrate commonly used methods of Java Collection Framework classes such as ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap and Hashtable.

Description

The program demonstrates collection classes by performing operations such as adding, accessing, updating, searching, removing, sorting, queue operations and key-value operations. It also demonstrates the use of Enumeration with Hashtable.

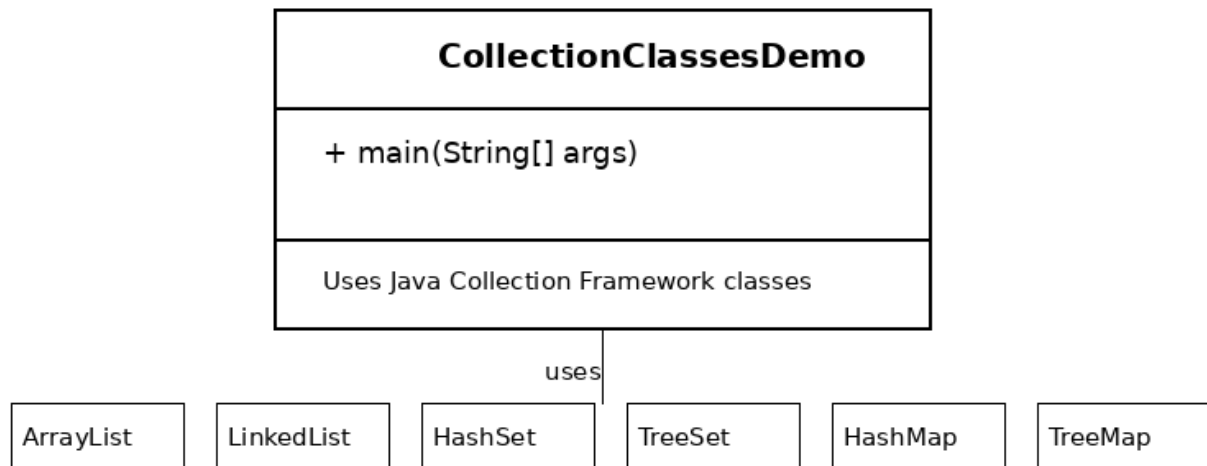
System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Algorithm

1. Start the program.
2. Create a Scanner object for user input.
3. Read two items from the user for ArrayList demonstration.
4. Create collection objects and apply the required methods for each class.
5. Demonstrate insertion, access, update, searching and removal operations on list classes.
6. Demonstrate LIFO operations using Stack.
7. Demonstrate unique-element operations using HashSet and LinkedHashSet.
8. Demonstrate sorted-set operations using TreeSet.
9. Demonstrate queue and deque operations using PriorityQueue and ArrayDeque.
10. Demonstrate key-value operations using HashMap, LinkedHashMap and TreeMap.
11. Demonstrate key, value and Enumeration operations using Hashtable.
12. Display the results of all operations.
13. Close Scanner and stop the program.



Source Code

```

import java.util.*;

public class CollectionClassesDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ===== ARRAYLIST =====
        System.out.println("===== ARRAYLIST =====");

        ArrayList<String> list = new ArrayList<>();

        System.out.print("Enter first item: ");
        String item1 = sc.nextLine();

        System.out.print("Enter second item: ");
        String item2 = sc.nextLine();

        list.add(item1);
        list.add(item2);
        list.add(1, "Java");

        System.out.println("ArrayList: " + list);
        System.out.println("Get index 1: " + list.get(1));

        list.set(1, "Python");
        System.out.println("After set: " + list);

        System.out.println("Contains Java: " + list.contains("Java"));
        System.out.println("Size: " + list.size());
        System.out.println("First index of item: " + list.indexOf(item1));
        System.out.println("Last index of item: " + list.lastIndexOf(item1));

        list.remove(item2);
        System.out.println("After remove: " + list);

        list.sort(Comparator.naturalOrder());
        System.out.println("Sorted: " + list);

        // ===== LINKEDLIST =====
        System.out.println("\n===== LINKEDLIST =====");

        LinkedList<String> linkedList = new LinkedList<>();

        linkedList.add("B");
        linkedList.addFirst("A");
        linkedList.addLast("C");
  
```

```

System.out.println("LinkedList: " + linkedList);
System.out.println("Get index 1: " + linkedList.get(1));
System.out.println("First: " + linkedList.getFirst());
System.out.println("Last: " + linkedList.getLast());

linkedList.offer("D");
System.out.println("After offer: " + linkedList);

System.out.println("Poll: " + linkedList.poll());
System.out.println("Peek: " + linkedList.peek());

linkedList.removeFirst();
linkedList.removeLast();

System.out.println("After removals: " + linkedList);

// ===== VECTOR =====
System.out.println("\n===== VECTOR =====");

Vector<String> vector = new Vector<>();

vector.add("Java");
vector.addElement("Python");
vector.add("C");

System.out.println("Vector: " + vector);
System.out.println("Get index 1: " + vector.get(1));

vector.set(1, "C++");
System.out.println("After set: " + vector);

System.out.println("Contains Java: " + vector.contains("Java"));
System.out.println("Size: " + vector.size());
System.out.println("Capacity: " + vector.capacity());

vector.remove(2);
vector.removeElement("Java");

System.out.println("After remove: " + vector);

// ===== STACK =====
System.out.println("\n===== STACK =====");

Stack<String> stack = new Stack<>();

stack.push("A");
stack.push("B");
stack.push("C");

System.out.println("Stack: " + stack);
System.out.println("Peek: " + stack.peek());
System.out.println("Search B: " + stack.search("B"));

System.out.println("Pop: " + stack.pop());
System.out.println("Stack after pop: " + stack);
System.out.println("Empty: " + stack.empty());

```

```
// ===== HASHSET =====
System.out.println("\n===== HASHSET =====");

HashSet<String> hashSet = new HashSet<>();

hashSet.add("Java");
hashSet.add("Python");
hashSet.add("Java");

System.out.println("HashSet: " + hashSet);
System.out.println("Contains Java: " + hashSet.contains("Java"));
System.out.println("Size: " + hashSet.size());

hashSet.remove("Python");
System.out.println("After remove: " + hashSet);

hashSet.clear();
System.out.println("After clear: " + hashSet);
System.out.println("Empty: " + hashSet.isEmpty());

// ===== LINKEDHASHSET =====
System.out.println("\n===== LINKEDHASHSET =====");

LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();

linkedHashSet.add("Java");
linkedHashSet.add("Python");
linkedHashSet.add("C");

System.out.println("LinkedHashSet: " + linkedHashSet);
System.out.println("Contains Java: "
    + linkedHashSet.contains("Java"));
System.out.println("Size: " + linkedHashSet.size());

linkedHashSet.remove("Python");
System.out.println("After remove: " + linkedHashSet);

linkedHashSet.clear();
System.out.println("After clear: " + linkedHashSet);

// ===== TREESET =====
System.out.println("\n===== TREESET =====");

TreeSet<Integer> treeSet = new TreeSet<>();

treeSet.add(30);
treeSet.add(10);
treeSet.add(20);
treeSet.add(40);

System.out.println("TreeSet: " + treeSet);
System.out.println("First: " + treeSet.first());
System.out.println("Last: " + treeSet.last());
System.out.println("Higher than 20: " + treeSet.higher(20));
System.out.println("Lower than 20: " + treeSet.lower(20));
System.out.println("Ceiling of 25: " + treeSet.ceiling(25));
System.out.println("Floor of 25: " + treeSet.floor(25));
```

```

System.out.println("Poll First: " + treeSet.pollFirst());
System.out.println("Poll Last: " + treeSet.pollLast());

System.out.println("After polling: " + treeSet);

// ===== PRIORITYQUEUE =====
System.out.println("\n===== PRIORITYQUEUE =====");

PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();

priorityQueue.add(30);
priorityQueue.offer(10);
priorityQueue.offer(20);

System.out.println("PriorityQueue: " + priorityQueue);
System.out.println("Peek: " + priorityQueue.peek());
System.out.println("Contains 20: " + priorityQueue.contains(20));
System.out.println("Size: " + priorityQueue.size());

System.out.println("Poll: " + priorityQueue.poll());
priorityQueue.remove(30);

System.out.println("After operations: " + priorityQueue);

// ===== ARRAYDEQUE =====
System.out.println("\n===== ARRAYDEQUE =====");

ArrayDeque<String> deque = new ArrayDeque<>();

deque.addFirst("B");
deque.addLast("C");
deque.offerFirst("A");
deque.offerLast("D");

System.out.println("ArrayDeque: " + deque);
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());

System.out.println("Poll First: " + deque.pollFirst());
System.out.println("Poll Last: " + deque.pollLast());

System.out.println("After polling: " + deque);

// ===== HASHMAP =====
System.out.println("\n===== HASHMAP =====");

HashMap<Integer, String> hashMap = new HashMap<>();

hashMap.put(101, "Rahul");
hashMap.put(102, "Priya");
hashMap.put(103, "Arjun");

System.out.println("HashMap: " + hashMap);
System.out.println("Value for 102: " + hashMap.get(102));
System.out.println("Contains key 101: "
    + hashMap.containsKey(101));
System.out.println("Contains Rahul: "

```

```

        + hashMap.containsKey("Rahul"));

System.out.println("Keys: " + hashMap.keySet());
System.out.println("Values: " + hashMap.values());
System.out.println("Entries: " + hashMap.entrySet());
System.out.println("Size: " + hashMap.size());

System.out.println("Unknown key: "
    + hashMap.getOrDefault(105, "Not Found"));

hashMap.remove(103);
System.out.println("After remove: " + hashMap);

// ===== LINKEDHASHMAP =====
System.out.println("\n===== LINKEDHASHMAP =====");

LinkedHashMap<Integer, String> linkedHashMap =
    new LinkedHashMap<>();

linkedHashMap.put(101, "Rahul");
linkedHashMap.put(102, "Priya");
linkedHashMap.put(103, "Arjun");

System.out.println("LinkedHashMap: " + linkedHashMap);
System.out.println("Value for 102: "
    + linkedHashMap.get(102));
System.out.println("Contains key 101: "
    + linkedHashMap.containsKey(101));
System.out.println("Keys: " + linkedHashMap.keySet());
System.out.println("Values: " + linkedHashMap.values());
System.out.println("Entries: " + linkedHashMap.entrySet());

linkedHashMap.remove(103);
System.out.println("After remove: " + linkedHashMap);

// ===== TREEMAP =====
System.out.println("\n===== TREEMAP =====");

TreeMap<Integer, String> treeMap = new TreeMap<>();

treeMap.put(30, "C");
treeMap.put(10, "A");
treeMap.put(20, "B");
treeMap.put(40, "D");

System.out.println("TreeMap: " + treeMap);
System.out.println("First Key: " + treeMap.firstKey());
System.out.println("Last Key: " + treeMap.lastKey());
System.out.println("Higher Key: " + treeMap.higherKey(20));
System.out.println("Lower Key: " + treeMap.lowerKey(20));
System.out.println("Ceiling Key: " + treeMap.ceilingKey(25));
System.out.println("Floor Key: " + treeMap.floorKey(25));
System.out.println("Entries: " + treeMap.entrySet());

treeMap.remove(30);
System.out.println("After remove: " + treeMap);

```

```

// ===== HASHTABLE =====
System.out.println("\n===== HASHTABLE =====");

Hashtable<Integer, String> hashtable = new Hashtable<>();

hashtable.put(101, "Rahul");
hashtable.put(102, "Priya");
hashtable.put(103, "Arjun");

System.out.println("Hashtable: " + hashtable);
System.out.println("Value for 102: " + hashtable.get(102));
System.out.println("Contains key 101: "
    + hashtable.containsKey(101));
System.out.println("Contains Rahul: "
    + hashtable.containsValue("Rahul"));

System.out.println("Keys:");
Enumeration<Integer> keys = hashtable.keys();

while (keys.hasMoreElements()) {
    System.out.println(keys.nextElement());
}

System.out.println("Values:");
Enumeration<String> values = hashtable.elements();

while (values.hasMoreElements()) {
    System.out.println(values.nextElement());
}

System.out.println("Size: " + hashtable.size());
System.out.println("Empty: " + hashtable.isEmpty());

hashtable.remove(103);
System.out.println("After remove: " + hashtable);

System.out.println("\n===== PROGRAM COMPLETED =====");

sc.close();
}
}

```

Compilation and Execution Commands

Save as: CollectionClassesDemo.java
 Compile: javac CollectionClassesDemo.java
 Execute: java CollectionClassesDemo

Output Screens

Possible Output – Collection Operations

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
===== ARRAYLIST =====  
Enter first item: DBMS  
Enter second item: CN  
ArrayList: [DBMS, Java, CN]  
Get index 1: Java  
After set: [DBMS, Python, CN]  
Contains Java: false  
Size: 3  
After remove: [DBMS, Python]
```

Possible Output – Map Operations

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
===== HASHMAP =====  
HashMap: {101=Rahul, 102=Priya, 103=Arjun}  
Value for 102: Priya  
Contains key 101: true  
Contains Rahul: true  
Keys: [101, 102, 103]  
Values: [Rahul, Priya, Arjun]  
Entries: [101=Rahul, 102=Priya, 103=Arjun]  
Size: 3
```

Result

Thus, the Java program was implemented successfully to demonstrate the important methods of the specified Java Collection Framework classes.